

EfficientDet-D0 Anomaly Detector: Evaluation Report

Checkpoints: models/efficient_det_export/best.pth (epoch 37) and last.pth (epoch 49) | evaluate.py, phi 0, --keep-empty
| 4 October 2026

Summary. The model is a good image-level defect detector on print types it has seen in training, and its boxes are only approximate. The test-split results (up to 100% accuracy) **overstate how good it is**. The validation split (about 95% accuracy, 89% recall, 97% specificity) is the more realistic estimate. **Recommended:** last.pth with a decision threshold of 0.25.

1. Data evaluated

Split	Images	Defective (has boxes)	Normal (no boxes)	Boxes
train	1856	912	944	1361
val	199	55	144	66
test	182	63	119	93

One class ("Anomaly"). Normal images have no annotations and act as negatives, so a detection on one counts as a false alarm.

2. Image-level results: "is this print defective?"

This is the metric behind the stop/continue decision. An image is flagged when its highest-scoring detection is at or above the decision threshold. Thresholds were **chosen on val and then applied to test**.

Ckpt	Split	Thresh	TP	FN	FP	TN	Recall	Specificity	Precision	ROC-AUC
best	val	0.20	49	6	4	140	0.891	0.972	0.925	0.932
last	val	0.25	49	6	4	140	0.891	0.972	0.925	0.940
best	test	0.20	61	2	1	118	0.968	0.992	0.984	0.999
last	test	0.25	63	0	0	119	1.000	1.000	1.000	1.000

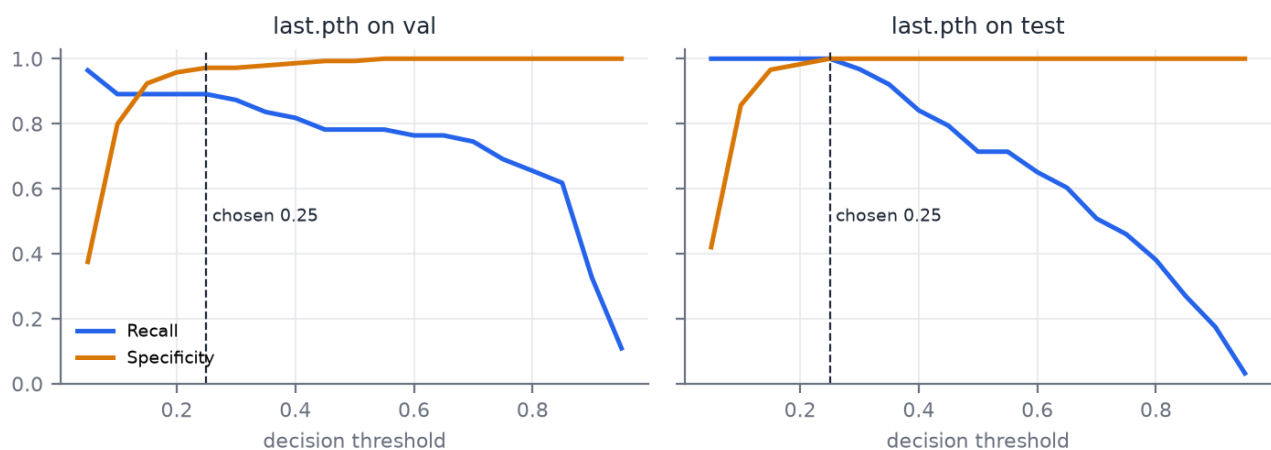


Figure 1. Recall and specificity as the decision threshold changes. The useful range is about 0.20-0.30. At the default --score-thresh of 0.05, roughly 60% of normal images get flagged.

Note: by default evaluate.py reports the best-F1 threshold *on the split being scored*. On test that tunes the threshold on the test set, which inflates the result (e.g. 0.15 gives 99.5% for best.pth). Pass --decision-thresh with a value chosen on val.

3. Box-level results (COCO mAP)

Ckpt	Split	AP@[.5:.95]	AP50	AP75	AP large	AP medium	AR@100
best	val	0.320	0.698	0.182	0.385	0.108	0.414
last	val	0.319	0.714	0.214	0.377	0.138	0.417
best	test	0.234	0.591	0.125	0.290	0.148	0.318
last	test	0.244	0.595	0.132	0.306	0.149	0.331

AP for small boxes is left out: val has 1 small box and test has 6, so those values are noise.

4. How to read the metrics

- **Recall:** the share of defective prints that get caught. A miss means a failed print keeps running. This is usually the number to protect.
- **Specificity:** the share of good prints left alone. A failure here is a false alarm that stops a good print.
- **Precision:** of the images flagged, the share that really were defective.
- **Ignore accuracy.** 65-72% of each split is normal, so a model that never flags anything already scores 65-72%.
- **Threshold:** raising it gives fewer false alarms and more missed defects. Scores are low because the model is trained with focal loss, which is why good thresholds sit around 0.2-0.3, not 0.5.
- **ROC-AUC:** how well the model ranks defective images above normal ones, regardless of threshold. 0.93-0.94 on val is solid.
- **AP50 vs AP75:** AP50 (about 0.6-0.7) means the model usually puts a box in the right region. AP75 (about 0.13-0.21) means the box edges are loose. Defects such as spaghetti or stringing have fuzzy, subjectively labelled edges, so a low AP@[.5:.95] is expected and doesn't matter much for a stop/continue decision.

5. Why test looks perfect and val doesn't

- **The test set is easier than val.** All of its defective images come from two sources (Roboflow images and success/failure images), and both are well represented in training. Val contains defect families that are almost absent from training: spiralanomaly, domeanomaly* and croppeddringanomaly (train has none of these, and only 3 chairanomaly images). Four of val's six misses come from these families, all scoring 0.00-0.07. The model hasn't generalized to object shapes it never saw.
- **All four val false alarms are "success (N).jpg" images** (scores 0.29-0.53): clean prints that look like failures.
- **Some leakage.** About 10% of val and test images (13/199 and 18/182) are near-duplicates of training images. The Ricardo-Metral-norm-frame-* images (608 train / 35 val / 15 test) are consecutive frames from one video, and they inflate specificity because every one is an easy true negative.
- **The test set is small.** "63/63 caught" still allows a true recall as low as about 95% at 95% confidence. Val's 49/55 gives a 95% confidence interval of roughly 78-95%.

Errors on val (last.pth, threshold 0.25), by image source

Source	TP	FN	FP	TN
success / failure	31	0	4	46
Roboflow (*.rf.*)	16	2	0	40
other (spiral, ring, chair...)	1	3	0	16
dome	1	1	0	3
Ricardo video frames	0	0	0	35

6. Checkpoint choice and training curve

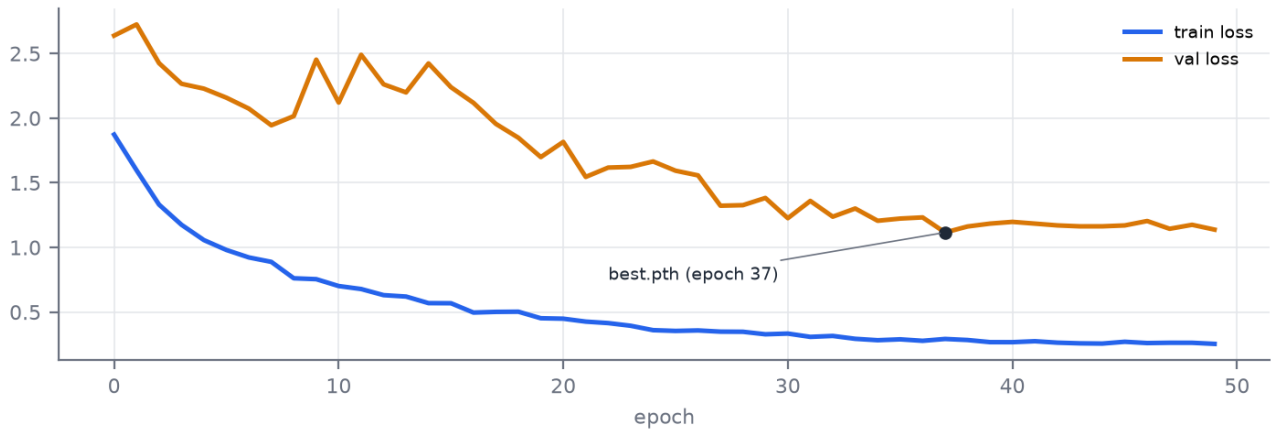


Figure 2. Training and validation loss per epoch.

- `last.pth` matches or beats `best.pth` on val (ROC-AUC 0.940 vs 0.932, AP75 0.214 vs 0.182) and on test, so val loss isn't a reliable way to pick the checkpoint here. Use `last.pth`, though the gap is within noise.
- Final train loss is 0.25 against val loss of 1.14, and val loss is flat from about epoch 37 onward. That's overfitting, which fits with about 1.9k images and a fully fine-tuned backbone (`FREEZE_BACKBONE=0`).
- Best to last was 12 epochs without improvement, so early stopping (`--patience`) wasn't active in this run.

7. Recommendations

- Deploy **`last.pth` with a decision threshold of 0.25**. When reporting test results, pass `--decision-thresh 0.25` instead of using the best-F1 threshold calculated on test.
- Split the data by **source or object family**, and keep video frames together, so held-out sets measure generalization rather than memorization.
- Add more training examples of dome, spiral, ring and chair defects, and of the success prints that look like failures.
- Try `FREEZE_BACKBONE=1` or the transfer mode to reduce overfitting, and compare on val.
- If missing a defect costs more than a false alarm, lower the threshold to about 0.15. On val that keeps recall at 0.89 and lets in 11 false alarms instead of 4 (7 with `best.pth`). Val's remaining misses score below 0.1, so going lower mainly adds false alarms.

Appendix: reproducing

```
python evaluate.py --images data/anomaly/images --ann
data/anomaly/annotations/instances_{val,test}.json --split all --keep-empty --phi 0 --checkpoint
models/efficient_det_export/{best,last}.pth --decision-thresh 0.25
```